

PROJECT REPORT

Smart Image Enhancement and Restoration System Using OpenCV

A Traditional Computer Vision Application

Submitted by
Abhay Mourya

Department of Computer Science and Engineering
VIT Bhopal University

Academic Project Report
2026

CERTIFICATE / DECLARATION

This project report presents the design and development of a Smart Image Enhancement and Restoration System using Python and OpenCV. The application demonstrates traditional computer vision operations through an interactive Streamlit interface.

I declare that this report has been prepared for academic submission and describes the implementation, methodology, features, and scope of the proposed system.

Student Signature	Faculty Signature
Abhay Mourya	

ABSTRACT

Digital images are frequently affected by inadequate lighting, low contrast, blur, and noise. The Smart Image Enhancement and Restoration System is designed to provide a simple interface for applying traditional image-processing techniques to improve visual quality. The system uses OpenCV, NumPy, Pillow, Matplotlib, and Streamlit to support image enhancement, restoration, comparison, and analysis. Users can perform grayscale conversion, brightness and contrast adjustment, histogram equalization, sharpening, embossing, edge detection, Gaussian blurring, median filtering, bilateral filtering, and denoising. The application also displays statistical information, histograms, Mean Squared Error (MSE), and Peak Signal-to-Noise Ratio (PSNR). The project focuses on interpretable and computationally accessible computer vision methods without using machine learning models.

Keywords

Computer Vision, OpenCV, Image Enhancement, Image Restoration, Histogram, Filtering, Streamlit

TABLE OF CONTENTS

Chapter	Topic	Page
1	Introduction	4
2	Problem Definition and Objectives	5
3	Background and Concepts	6
4	System Requirements and Architecture	7
5	Methodology and Algorithms	8
6	Implementation Details	10
7	Testing and Evaluation	11
8	Applications, Limitations and Future Scope	12
9	Conclusion	13
10	References	13

1. INTRODUCTION

1.1 Overview

Image processing is an important area of computer vision that focuses on improving, transforming, and analyzing digital images. Images captured using cameras, mobile devices, scanners, or other sensors may contain noise, blur, poor illumination, and contrast-related problems. Image enhancement and restoration techniques are used to address these issues and make visual information easier to interpret.

1.2 Project Overview

This project implements a web-based image-processing application using Streamlit. The user uploads an image and selects an operation from enhancement or restoration categories. The application processes the image and presents the output along with a comparison view and analytical information.

1.3 Scope

- Processing of individual images through an interactive interface.
- Application of traditional, non-learning-based image-processing algorithms.
- Visual comparison between original and processed images.
- Statistical and histogram-based image analysis.

2. PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Definition

Many images require preprocessing before they can be viewed, analyzed, or used in further applications. Existing professional editing tools may be complex for beginners and may not clearly demonstrate the underlying computer vision concepts. The proposed system addresses this issue by combining common image enhancement and restoration operations in one accessible interface.

2.2 Objectives

- Improve brightness and contrast in digital images.
- Convert images into grayscale for simplified analysis.
- Enhance edges and fine details using sharpening.
- Apply histogram equalization to improve intensity distribution.
- Detect image edges using the Canny edge detector.
- Reduce noise and blur using multiple restoration filters.
- Provide statistical measurements and visual histograms.
- Allow users to download the processed image.

2.3 Expected Outcome

The expected outcome is a functional and user-friendly application that demonstrates how traditional computer vision methods affect image appearance and measurable image properties.

3. BACKGROUND AND CONCEPTS

3.1 Digital Image Representation

A digital image is represented as a matrix of pixels. A grayscale image generally contains one intensity value per pixel, while a color image commonly contains three channels representing blue, green, and red values in OpenCV's default representation. Pixel values are typically stored in the range 0 to 255 for 8-bit images.

3.2 Convolution and Filtering

Convolution applies a small matrix called a kernel to neighboring pixels. Different kernels produce different effects, such as smoothing, sharpening, embossing, or edge emphasis. Filtering is a core operation in image enhancement and restoration.

3.3 Histograms

An image histogram represents the frequency of intensity values. Histogram analysis helps identify whether an image is dark, bright, low contrast, or broadly distributed across the available intensity range.

3.4 Quality Metrics

Mean Squared Error measures the average squared difference between two images. PSNR is calculated from MSE and is commonly used as a signal-quality indicator. These metrics must be interpreted with the processing objective because a visually useful transformation may intentionally change pixel values.

4. SYSTEM REQUIREMENTS AND ARCHITECTURE

4.1 Hardware Requirements

- Computer or laptop with a modern processor.
- Minimum 4 GB RAM; 8 GB or more is recommended.
- Sufficient storage for Python packages and image files.
- Display capable of running a modern web browser.

4.2 Software Requirements

- Python 3.x
- OpenCV
- NumPy
- Pillow
- Matplotlib
- Streamlit
- scikit-image
- streamlit-image-comparison

4.3 System Workflow

The system follows this workflow: image upload → validation and conversion → operation selection → image processing → result visualization → statistical analysis → optional download.

4.4 Modular Architecture

The implementation is divided into modules. Enhancement functions are stored separately from restoration functions and analysis functions. This modular structure improves maintainability, testing, and future expansion.

5. METHODOLOGY AND ALGORITHMS

5.1 Enhancement Algorithms

- **Grayscale Conversion:** Converts a three-channel image into a single-channel intensity image.
- **Brightness Adjustment:** Adds an intensity offset to increase or decrease brightness.
- **Contrast Adjustment:** Multiplies pixel intensities by a contrast factor.
- **Histogram Equalization:** Redistributes grayscale intensities to improve contrast.
- **Sharpening:** Uses a high-frequency convolution kernel to emphasize details.
- **Embossing:** Uses a directional kernel to create a relief-like appearance.
- **Canny Edge Detection:** Detects strong intensity boundaries after grayscale conversion.

5.2 Restoration Algorithms

- **Gaussian Blur:** Smooths pixels using weighted neighborhood averaging.
- **Median Filter:** Replaces a pixel using the median of nearby values and is useful for impulse noise.
- **Bilateral Filter:** Smooths an image while preserving significant edges.
- **Non-Local Means Denoising:** Reduces noise by using similarities between image regions.

5.3 Processing Parameters

Some operations use adjustable parameters such as brightness value, contrast factor, filter kernel size, and denoising strength. The selected values influence the visual result and should be chosen according to the image and desired outcome.

6. IMPLEMENTATION DETAILS

6.1 Project Structure

```
Smart-Image-Enhancement/  
■■■ app.py  
■■■ README.md  
■■■ statement.md  
■■■ requirements.txt  
■■■ modules/  
■ ■■■ __init__.py  
■ ■■■ enhancement.py  
■ ■■■ restoration.py  
■ ■■■ analysis.py  
■■■ utils/  
■ ■■■ __init__.py  
■ ■■■ image_utils.py  
■■■ tests/  
■ ■■■ test_processing.py  
■■■ sample_images/
```

6.2 Application Interface

The Streamlit interface contains an image uploader, sidebar operation categories, processing controls, result visualization, image comparison, download functionality, and analysis sections. The interface is designed to make the processing pipeline understandable to users.

6.3 Error Handling

The application validates whether an image has been uploaded and restricts excessively large files. The system also handles grayscale and color outputs separately when preparing images for display and analysis.

6.4 Output Generation

Processed images can be displayed in the browser and exported as PNG files. The application also records processing history during the current session for user reference.

7. TESTING AND EVALUATION

7.1 Testing Strategy

Testing focuses on verifying that the application accepts valid images, applies each operation, displays the output, and provides analysis information. Each operation should be tested using images with different brightness, contrast, texture, and noise characteristics.

Test Case	Expected Result	Status
Upload valid image	Image is displayed correctly	To be verified
Apply grayscale	Single-channel grayscale output	To be verified
Adjust brightness	Image becomes brighter or darker	To be verified
Adjust contrast	Contrast changes according to factor	To be verified
Apply sharpening	Details and edges are emphasized	To be verified
Apply Gaussian blur	Image becomes smoother	To be verified
Apply median filter	Impulse noise is reduced	To be verified
Download result	PNG file is generated	To be verified

7.2 Evaluation Metrics

The application uses mean intensity, minimum intensity, maximum intensity, standard deviation, histograms, MSE, and PSNR to support evaluation. These measurements provide numerical context for the visual changes observed after processing.

7.3 Results Documentation

For the final academic submission, screenshots of the interface, enhancement outputs, restoration outputs, and histogram analysis can be inserted in this section as evidence of execution.

8. APPLICATIONS, LIMITATIONS AND FUTURE SCOPE

8.1 Applications

- Basic photo enhancement and preprocessing.
- Educational demonstrations of filtering and convolution.
- Preparation of images for computer vision pipelines.
- Noise reduction in scanned or captured images.
- Visual inspection of image intensity and contrast.

8.2 Limitations

- The system does not automatically select the best algorithm for every image.
- Output quality depends on parameter selection.
- Traditional filters may remove useful details along with noise.
- The current application primarily processes one image at a time.
- MSE and PSNR alone cannot fully represent human visual quality.

8.3 Future Scope

- Add batch image processing.
- Provide preset enhancement profiles.
- Include additional restoration algorithms.
- Add image-quality charts and downloadable reports.
- Support side-by-side comparison of multiple processing methods.
- Deploy the application as a publicly accessible web service.

9. CONCLUSION

The Smart Image Enhancement and Restoration System demonstrates the practical application of traditional computer vision techniques using OpenCV. The system integrates enhancement, restoration, visual comparison, and statistical analysis within an accessible Streamlit interface. It provides a useful learning platform for understanding grayscale conversion, histogram processing, convolution, filtering, edge detection, and image-quality evaluation. The modular design also allows future improvements such as batch processing, advanced controls, and expanded restoration methods.

10. REFERENCES

- OpenCV Documentation: <https://docs.opencv.org/>
- Python Documentation: <https://docs.python.org/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- NumPy Documentation: <https://numpy.org/doc/>
- Matplotlib Documentation: <https://matplotlib.org/>

— End of Report —